

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Partea a III-a

Faza de proiectare - Diagrame de Secvență

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Diagrame de Secvență	2
1.1	Utilizator Neautentificat	2
1.2	Autentificare și Delogare (Comun)	4
1.3	Utilizator tip Angajat	6
1.4	Utilizator tip Manager	14
1.5	Utilizator tip Administrator	20

1 Diagrame de Secvență

Diagramele de secvență detaliază interacțiunile dinamice dintre componentele sistemului, ilustrând fluxul temporal al apelurilor de la acțiunea utilizatorului în interfața grafică (JavaFX), traversând straturile arhitecturii MVVM, trecând prin infrastructura de rețea (API Gateway) și ajungând la microserviciile responsabile cu logica de business și persistența datelor.

1.1 Utilizator Neautentificat

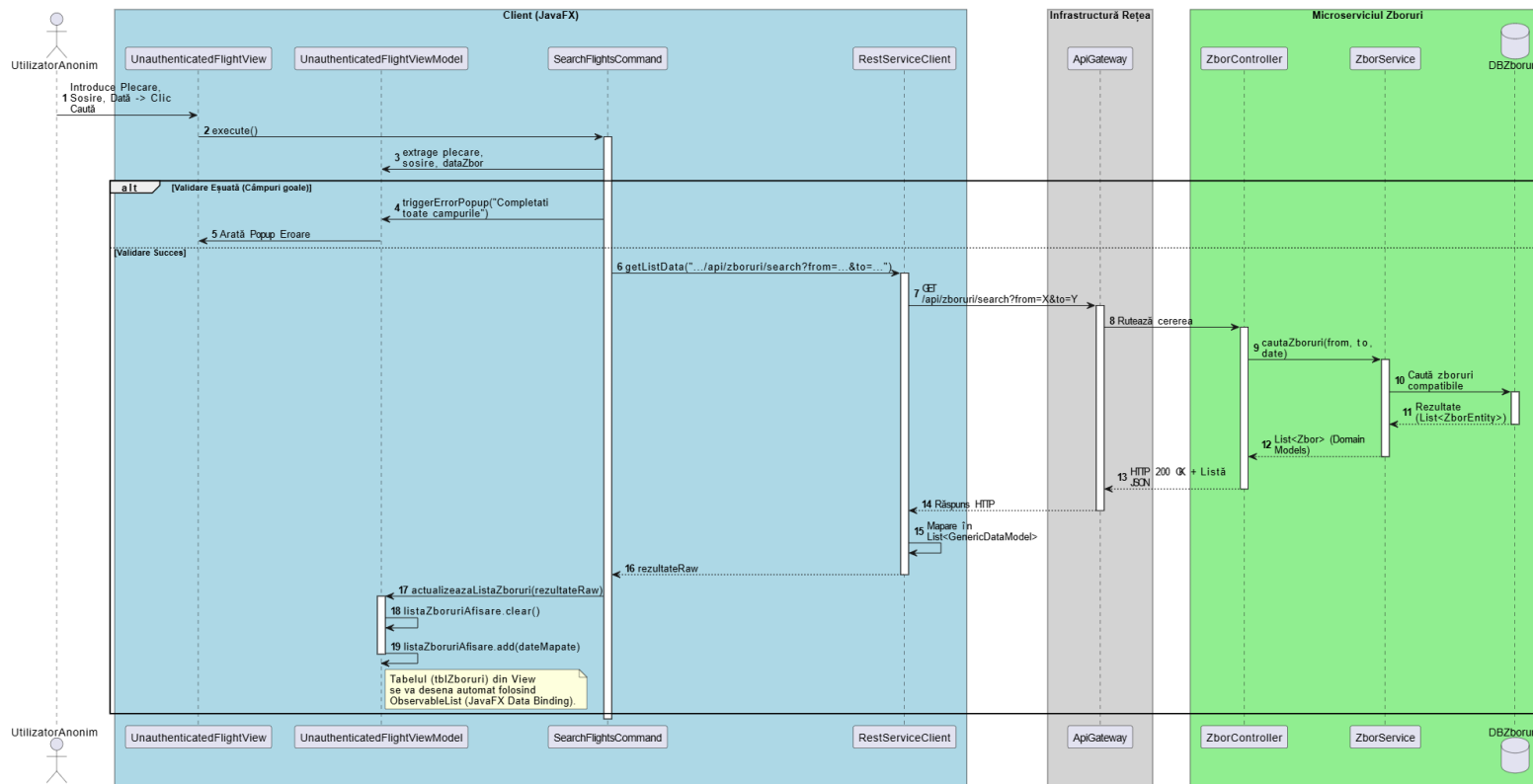


Figura 1: Diagrama de secvență: Căutare Zboruri

Descrierea fluxului: Utilizatorul anonim introduce datele de căutare (aeroporturi, dată) și apasă butonul „Caută”. `SearchFlightsCommand` validează câmpurile local. Dacă sunt corecte, folosește `RestServiceClient` pentru a iniția un apel `GET` către API Gateway, care rutează cererea la microserviciul `Zboruri`. Aici,

ZborService interoghează baza de date, obținând lista de entități compatibile, pe care o returnează sub formă de JSON. La întoarcere, clientul mapează JSON-ul și actualizează structura **ObservableList**, ceea ce declanșează actualizarea automată a tabelului din interfața utilizatorului prin Data Binding.

1.2 Autentificare și Delogare (Comun)

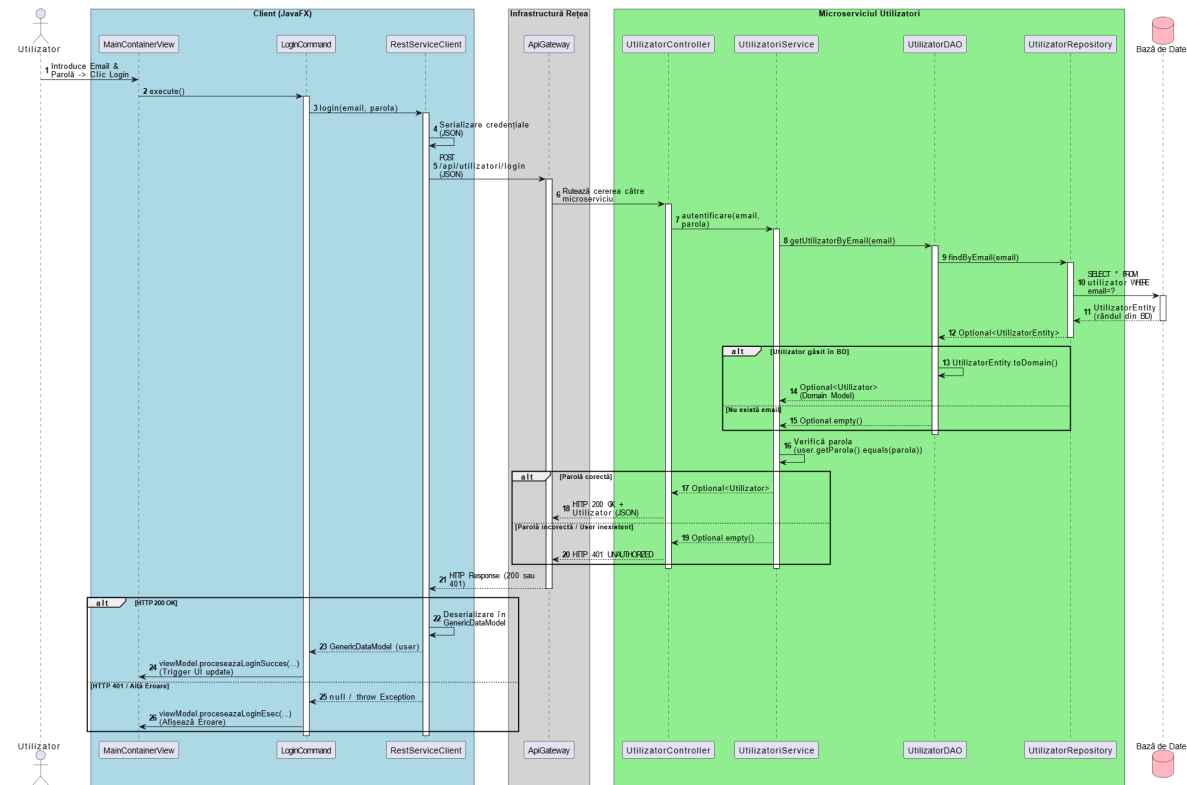


Figura 2: Diagrama de secvență: Autentificare utilizator

Descrierea fluxului: Pentru autentificare, datele introduse de utilizator sunt serializate în JSON de către comandă și trimise printr-un apel POST către Microserviciul Utilizatori. UtilizatoriService utilizează DAO pentru a căuta emailul în baza de date. Dacă entitatea este găsită, se compară parola. Microserviciul returnează HTTP 200 OK împreună cu obiectul utilizatorului, sau HTTP 401 Unauthorized în caz de eșec. În funcție de statusul HTTP, MainContainerViewModel actualizează starea aplicației, dând acces la modulele specifice rolului.

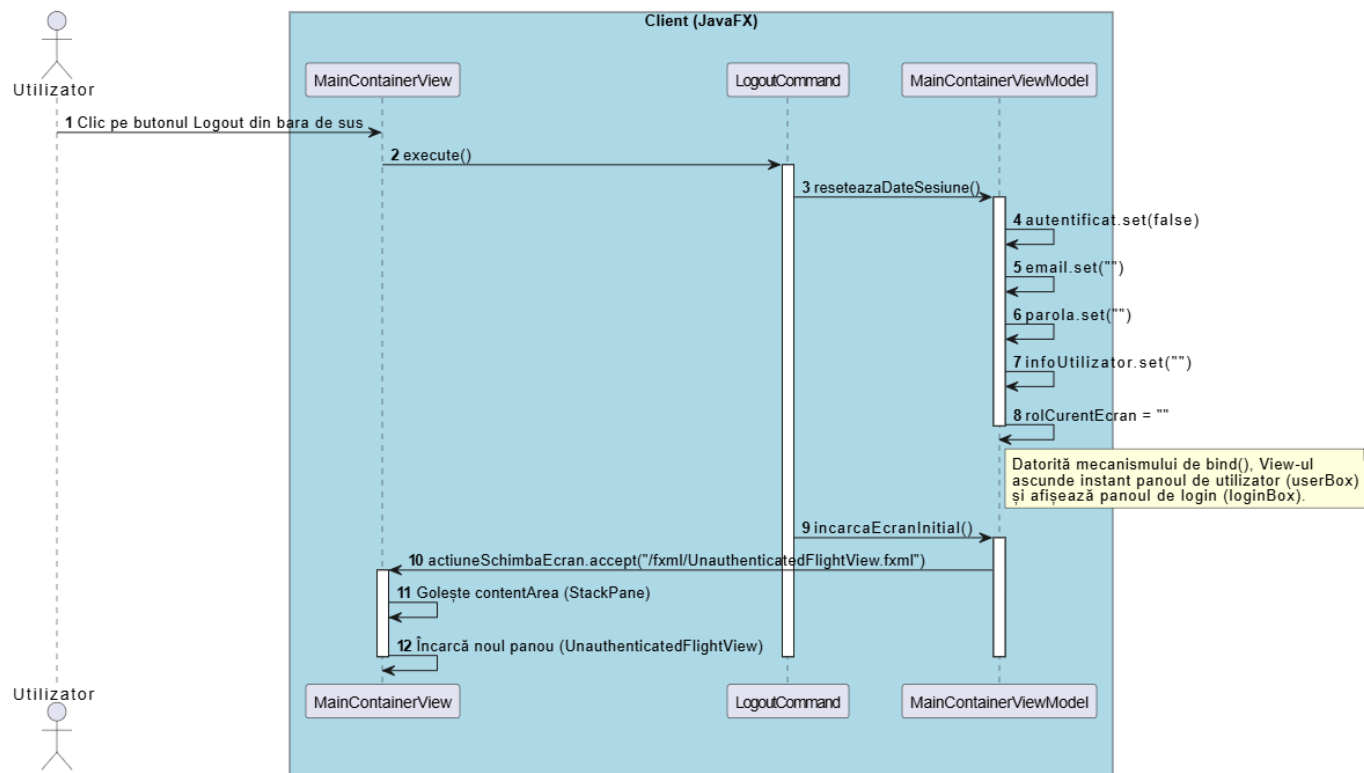


Figura 3: Diagrama de secvență: Delogare utilizator

Descrierea fluxului: Delogarea este un proces care se desfășoară exclusiv la nivelul clientului. La apăsarea butonului de *Logout*, *LogoutCommand* accesează *MainContainerViewModel* și resetează toate datele sesiunii (statusul de autentificare trece pe **false**, parolele și detaliile utilizatorului sunt golite). Prin mecanismul intern al JavaFX, view-ul ascunde instantaneu panoul destinat utilizatorilor autentificați și reîncarcă interfața publică implicită.

1.3 Utilizator tip Angajat

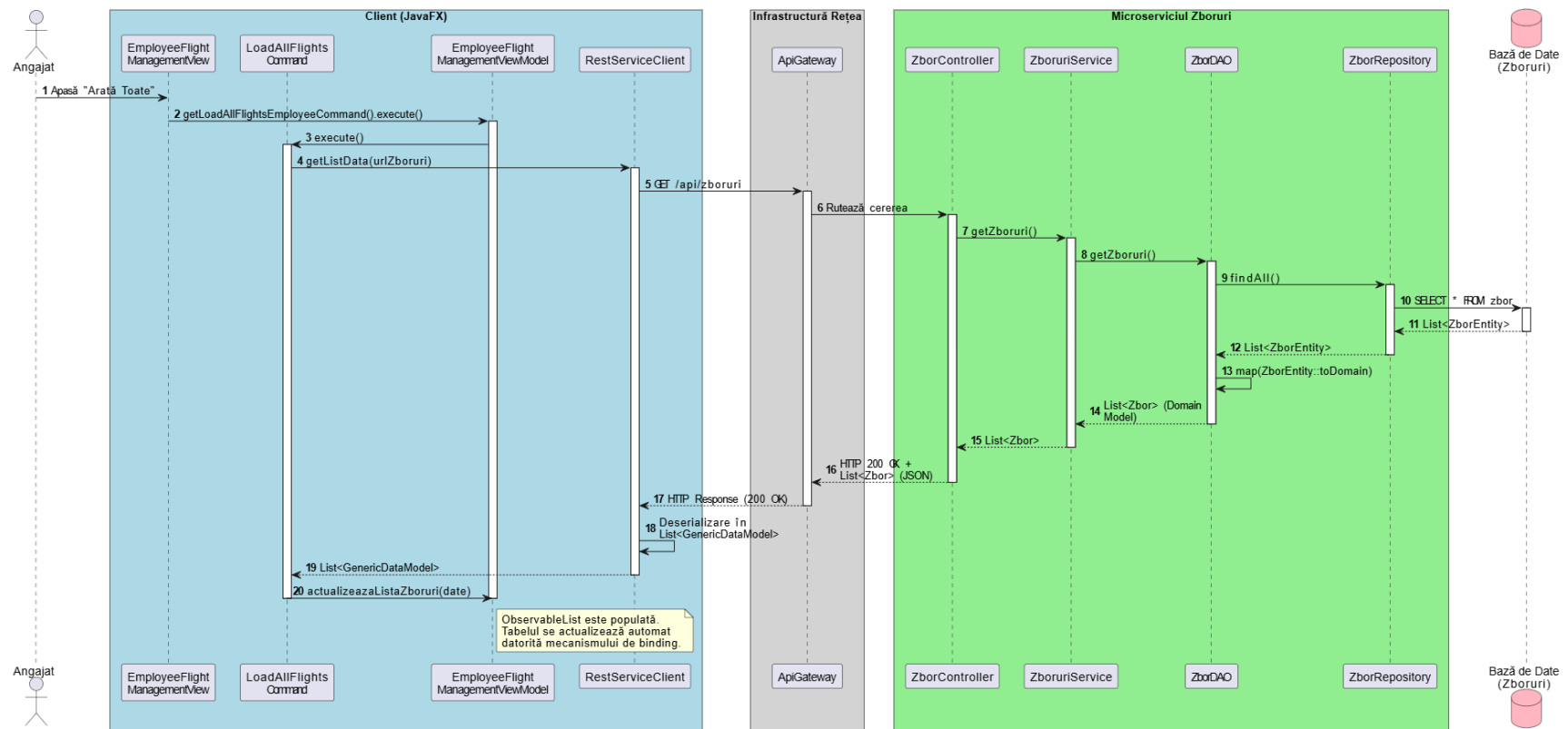


Figura 4: Diagrama de secvență: Vizualizarea listei de zboruri

Descrierea fluxului: Angajatul apasă butonul „Arată Toate” pentru a încărca zborurile disponibile. LoadAllFlightsCommand inițiază un apel de tip GET către Microserviciul Zboruri prin intermediul API Gateway. Backend-ul interoghează baza de date, extrage lista entităților de zbor și o returnează clientului sub formă de array JSON. Clientul deserializează răspunsul și actualizează structura ObservableList din EmployeeFlightManagementViewModel. Datorită mecanismului intern de Data Binding din JavaFX, tabelul din interfața grafică se populează automat, fără a fi nevoie de o actualizare manuală a elementelor de UI.

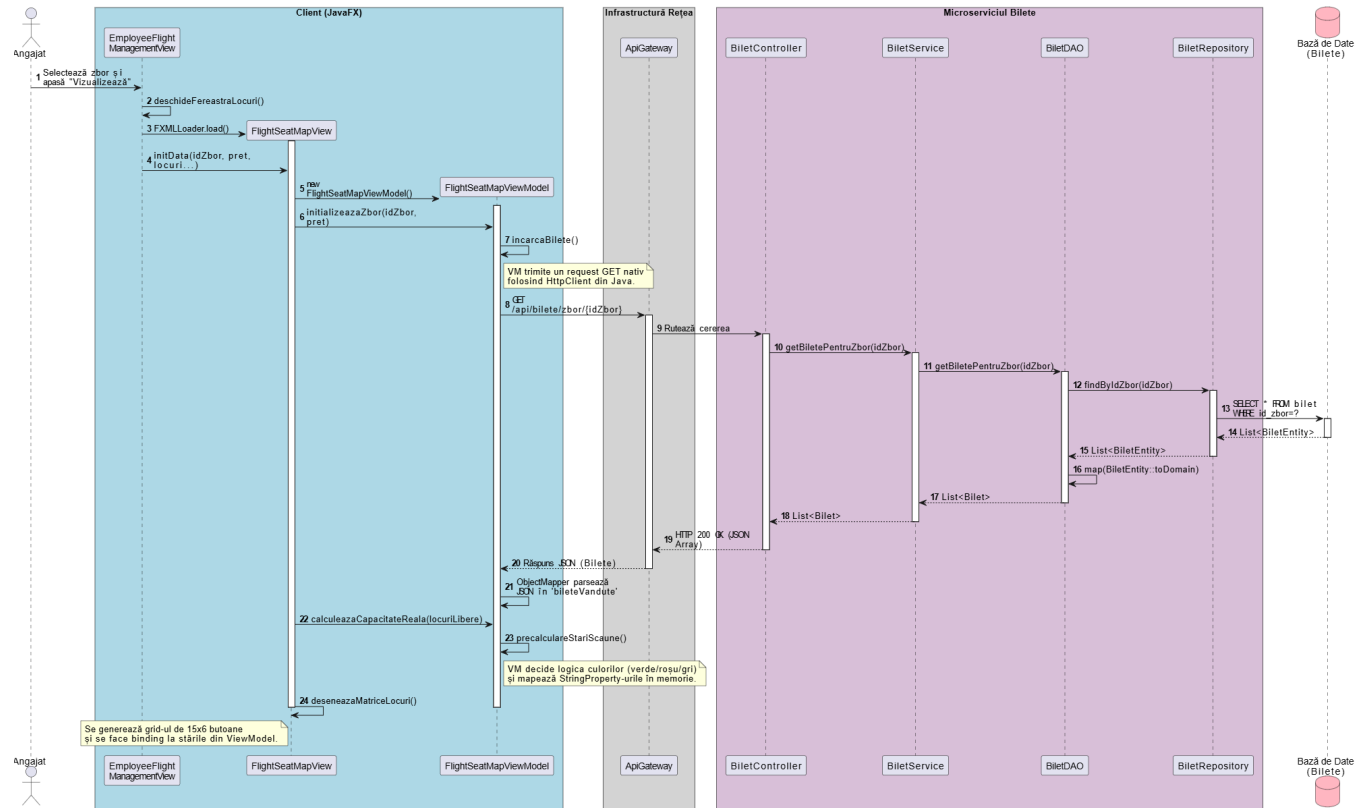


Figura 5: Diagrama de secvență: Inițializarea și vizualizarea hărții locurilor

Descrierea fluxului: La deschiderea ferestrei cu harta locurilor pentru un zbor, ViewModel-ul face o cerere **GET** către microserviciul **Bilete** pentru a prelua toate biletele vândute aferente acelui zbor. JSON-ul primit este parsat, iar aplicația precalculează stările scaunelor. Se generează dinamic matricea (grid-ul) de butoane în **FlightSeatMapView**, fiecare buton fiind conectat (binded) la starea din memorie (liber - verde, ocupat - roșu).

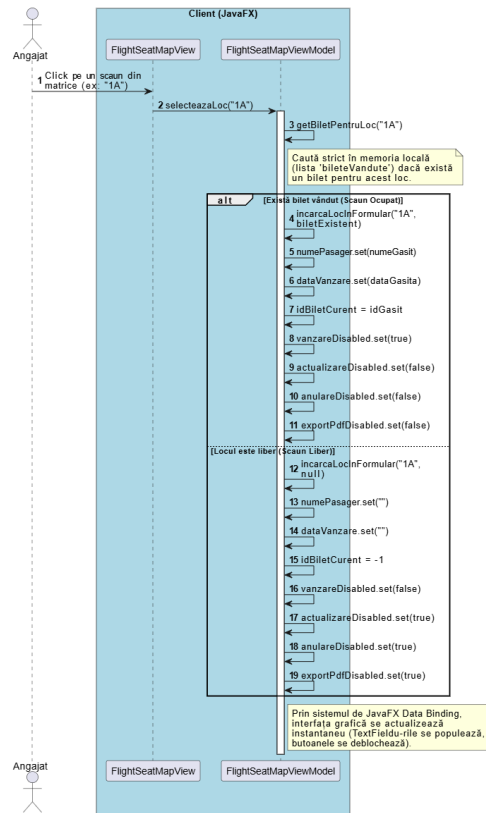


Figura 6: Diagrama de secvență: Interacțiunea cu un scaun din avion (Local)

Descrierea fluxului: Când angajatul face click pe un scaun (ex: "1A"), acțiunea nu declanșează un apel de rețea, ci o căutare în lista locală de bilete descărcată anterior. Dacă locul este ocupat, formularul se populează cu numele pasagerului și data vânzării, activând butoanele de *Actualizare*, *Anulare* și *Export PDF*. Dacă locul este liber, formularul se golește și se activează doar butonul de *Vânzare*.

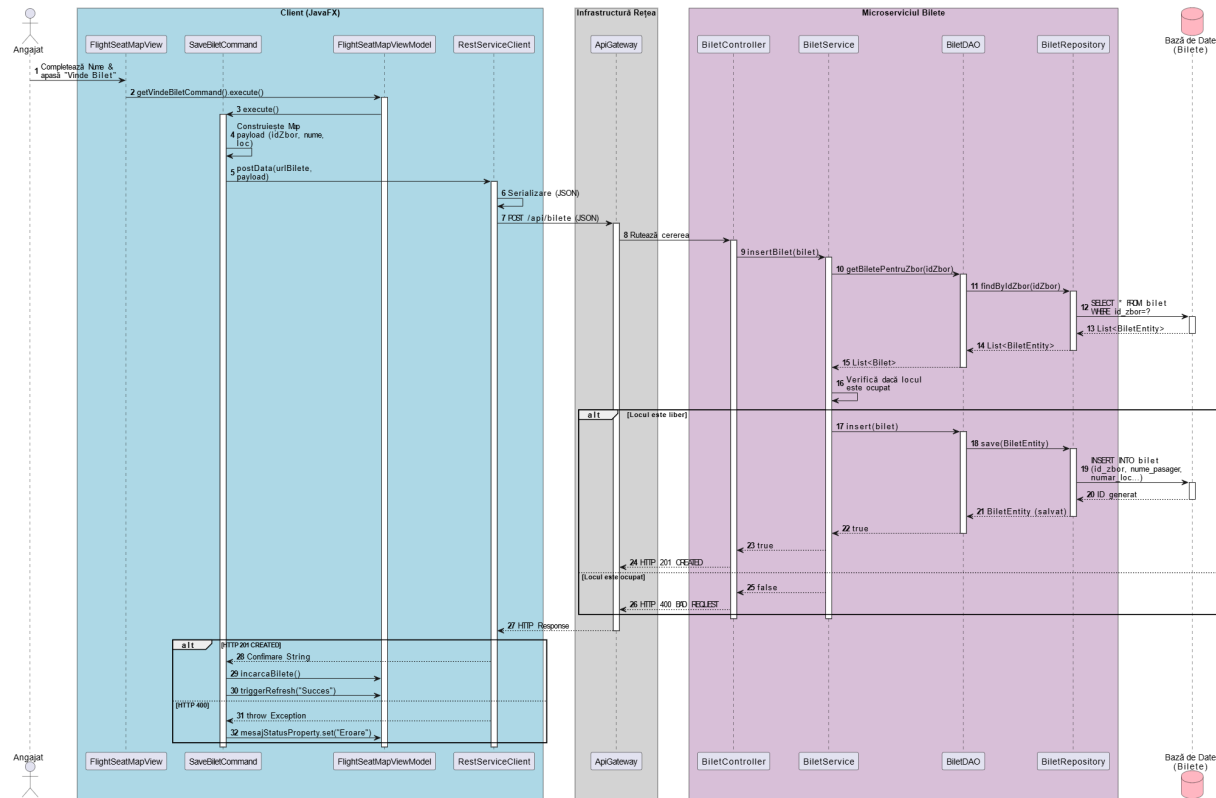


Figura 7: Diagrama de secvență: Vânzarea (adăugarea) unui bilet

Descrierea fluxului: După completarea formularului pentru un scaun liber, se declanșează **SaveBiletCommand**. Se construiește payload-ul și se trimite ca POST spre **Microserviciul Bilete**. Backend-ul face o ultimă verificare în baza de date pentru a garanta că locul nu a fost ocupat concurrent între timp. Dacă este liber, îl salvează și întoarce HTTP 201 Created. Clientul notifică succesul și reîncarcă harta locurilor.

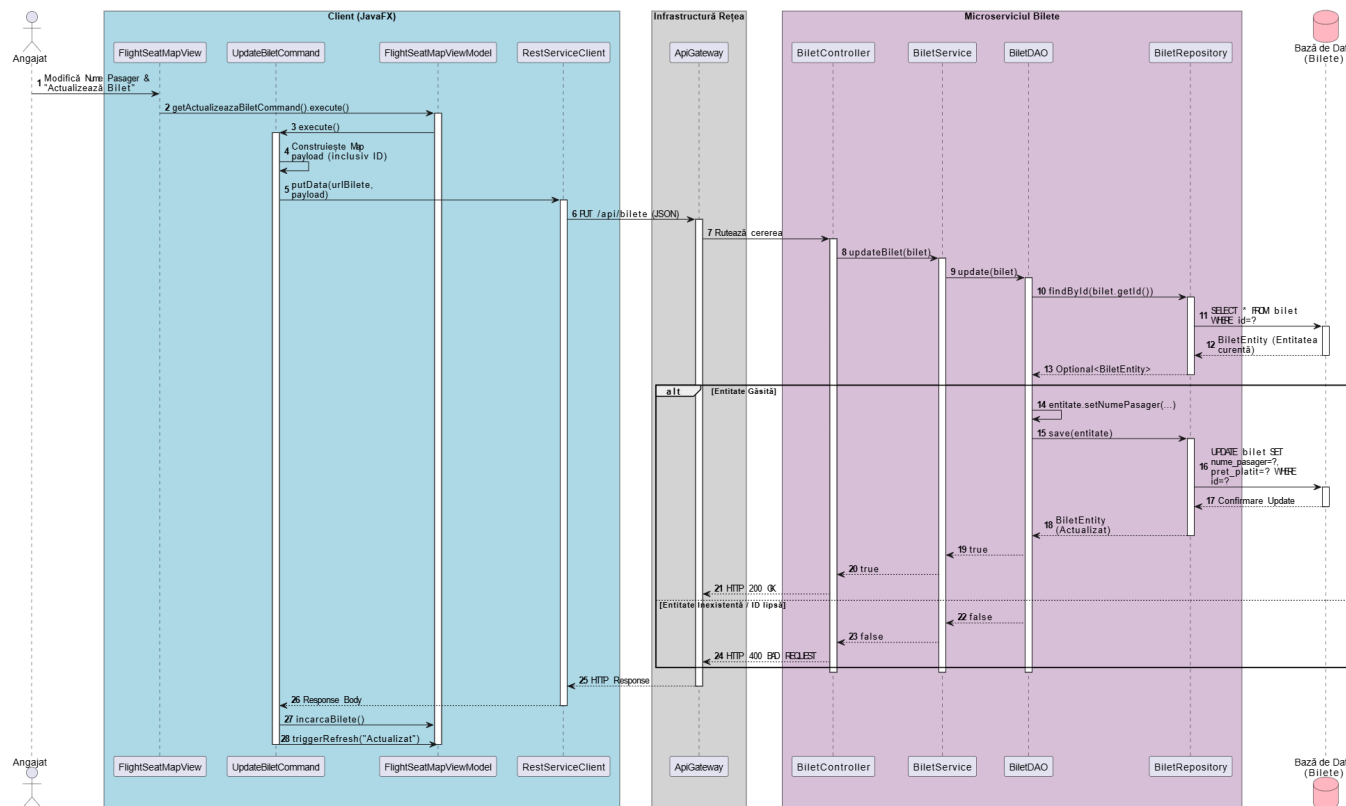


Figura 8: Diagrama de secvență: Actualizarea unui bilet existent

Descrierea fluxului: Angajatul modifică numele pasagerului și apasă actualizare. Sistemul formează o cerere PUT ce include ID-ul biletului. **BiletService** folosește DAO pentru a încărca entitatea existentă, aplică noile date și persista modificările (UPDATE). Răspunsul HTTP 200 OK determină aplicația client să dea un refresh vizual.

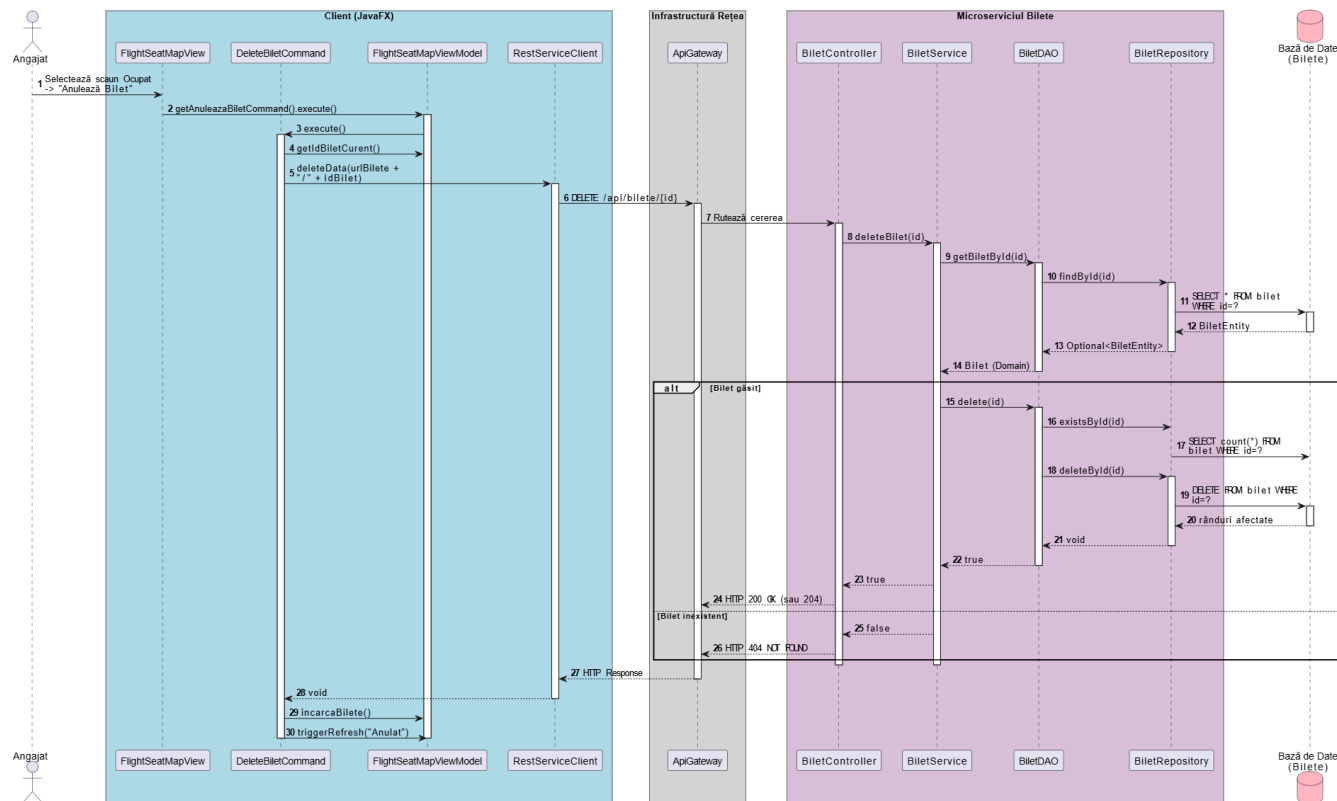


Figura 9: Diagrama de secvență: Anularea (ștergerea) unui bilet

Descrierea fluxului: Anularea rezervării inițiază un apel de tip DELETE cu ID-ul biletului în calea URL-ului (`/api/bilete/{id}`). Microserviciul verifică existența biletului, iar ulterior șterge rândul din baza de date folosind repository-ul JPA. La primirea răspunsului de succes, locul redevine verde și disponibil pentru vânzare în UI.

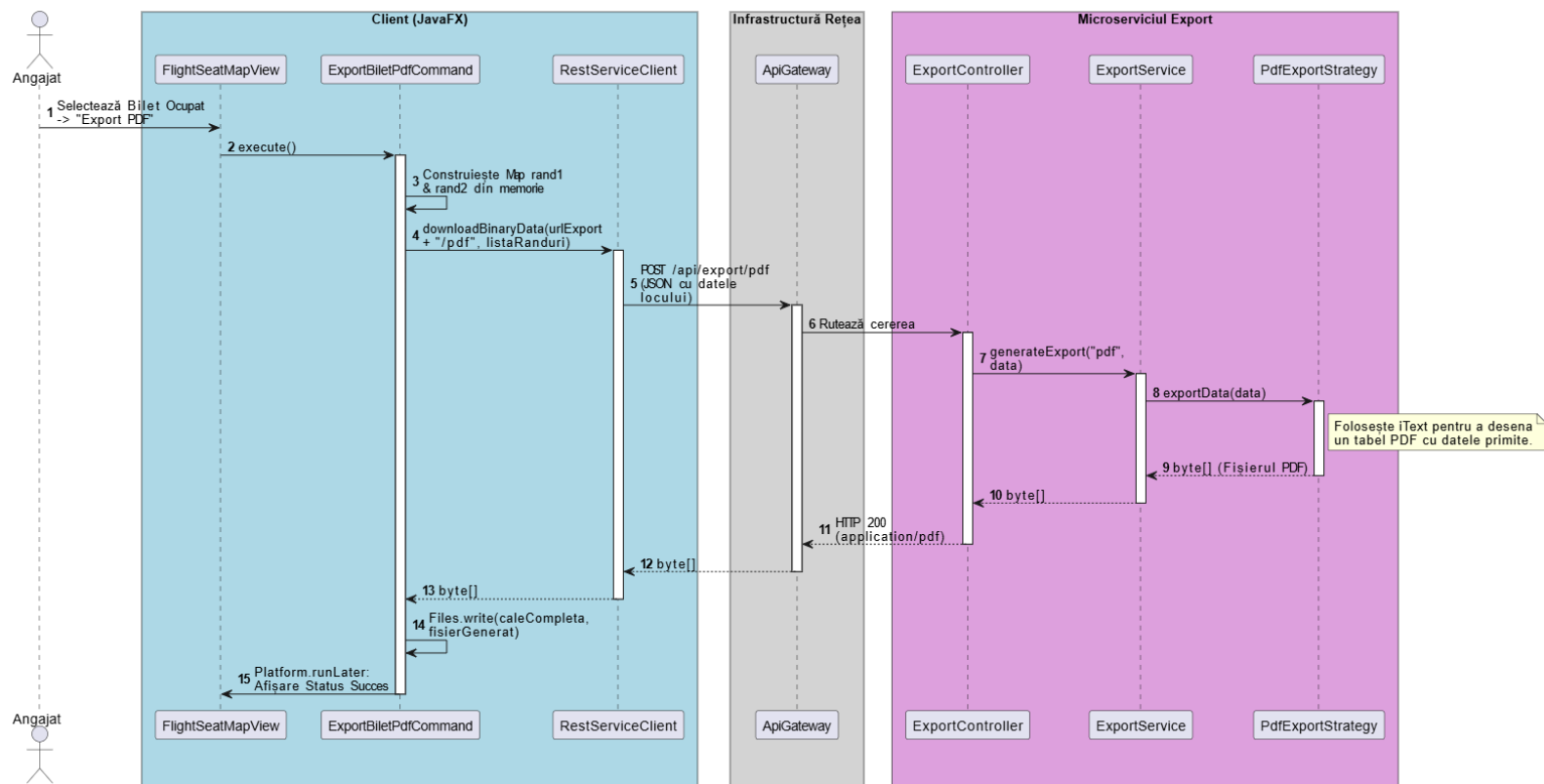


Figura 10: Diagrama de secvență: Exportul unui singur bilet în format PDF

Descrierea fluxului: Generarea unui document se realizează pe server pentru a decupla logica de rendering de aplicația client. Informațiile biletului sunt trimise ca POST către Microserviciul Export. Aici, `ExportService` utilizează o strategie (folosind o bibliotecă precum `iText`) pentru a desena un tabel PDF cu datele primite. Documentul generat este returnat clientului sub formă de array de octeți (`byte[]`), pe care comanda JavaFX îl scrie pe discul local al utilizatorului.

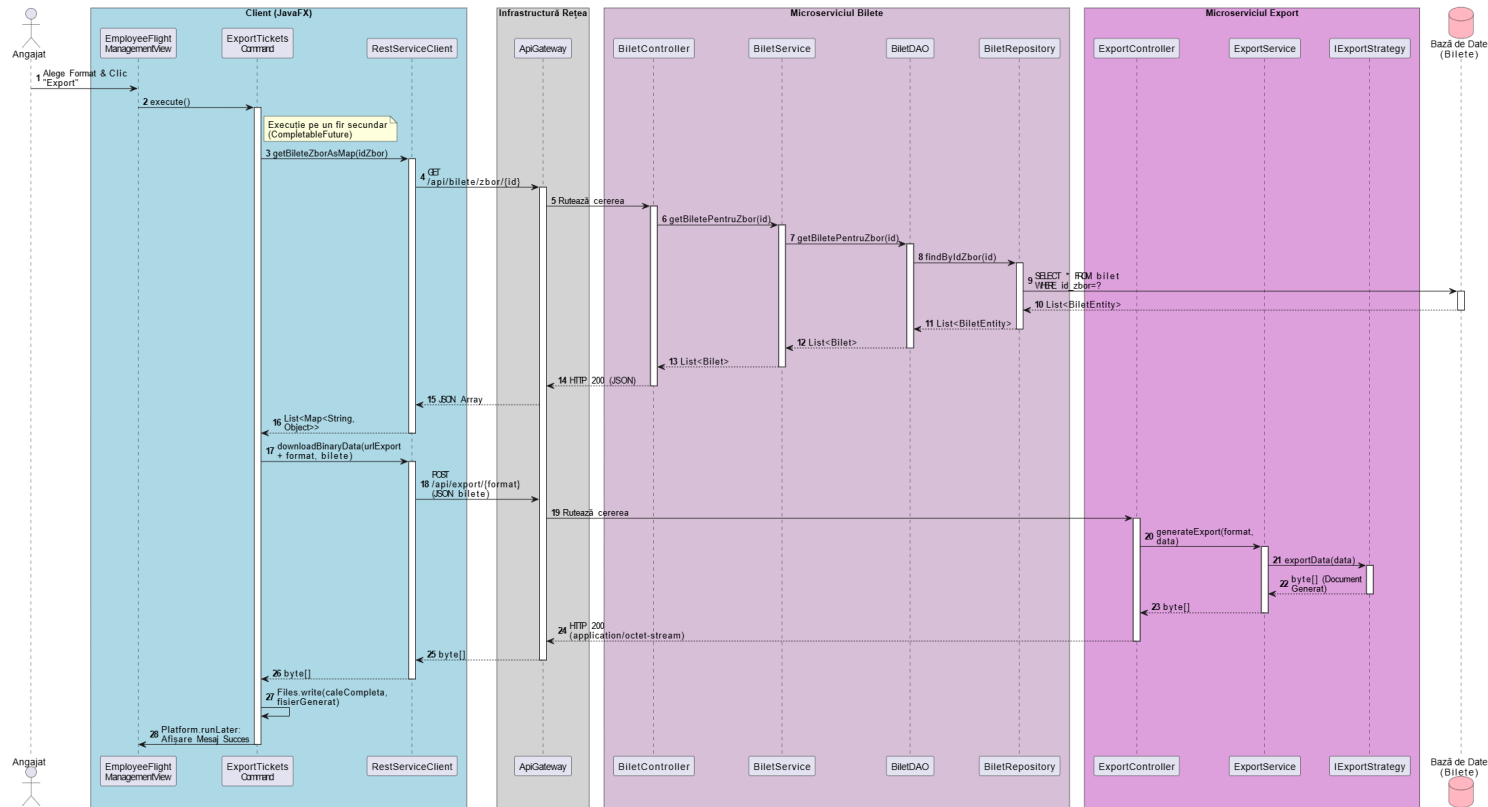


Figura 11: Diagrama de secvență: Exportul listei tuturor biletelor (Raport)

Descrierea fluxului: Acest proces implică două microservicii. Mai întâi, clientul preia lista completă a biletelor pentru zborul selectat apelând `GET` la `Microserviciul Bilete`. Apoi, trimite întregul Array de date JSON către `Microserviciul Export` printr-un `POST`. Similar cazului anterior, raportul final (PDF, CSV etc., în funcție de formatul cerut) este asamblat de server și livrat în format binar (`application/octet-stream`) clientului, care îl stochează local și notifică angajatul.

1.4 Utilizator tip Manager

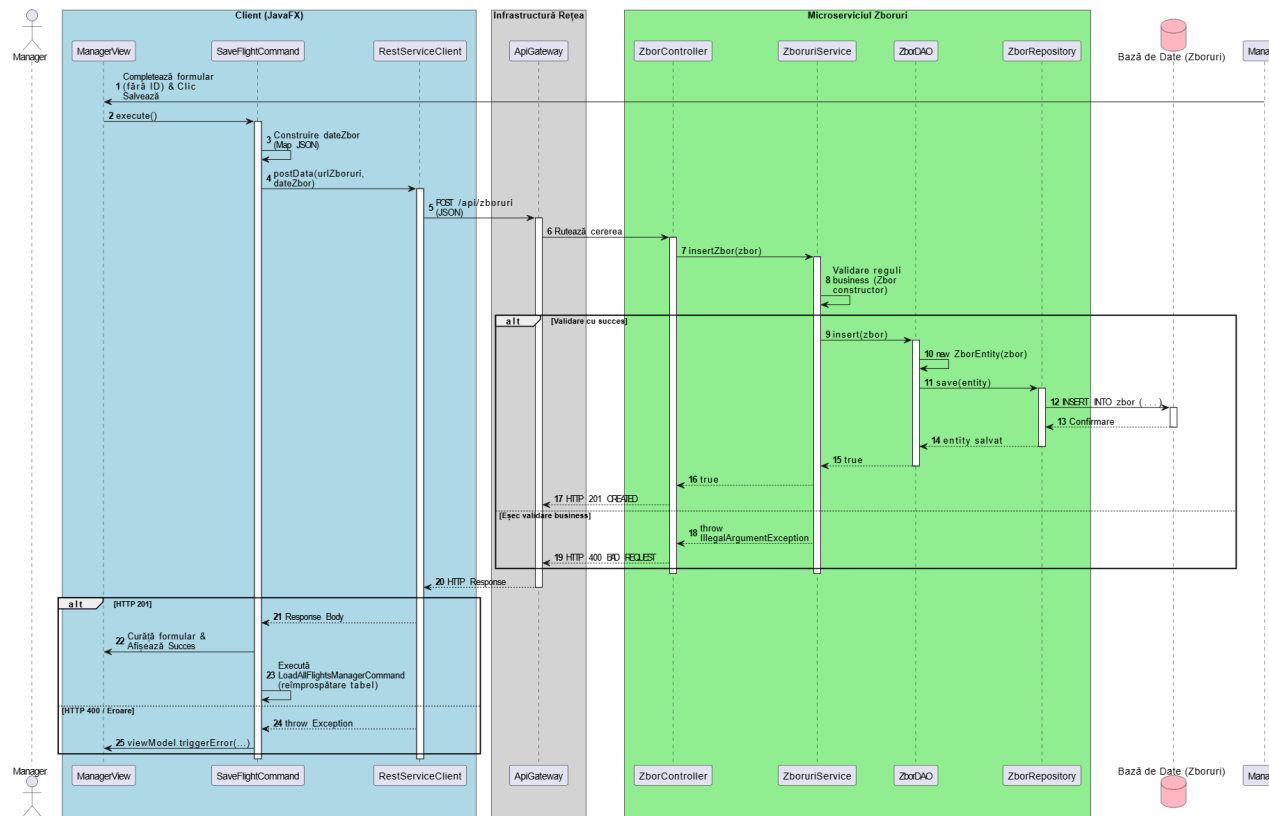


Figura 12: Diagrama de secvență: Adăugarea unui zbor nou

Descrierea fluxului: Managerul completează datele zborului în formular și apasă „Salvează”. Deoarece nu există un ID selectat, `SaveFlightCommand` serializează datele și inițiază un apel POST către `Microserviciul Zboruri`. În backend, serviciul validează regulile de business (ex. capacitate validă, aeroporturi distincte). Dacă validarea trece, entitatea este salvată în baza de date (INSERT). La primirea răspunsului HTTP 201 Created, clientul curăță formularul și apelează comanda de reîncărcare a tabelului pentru a afișa noul zbor.

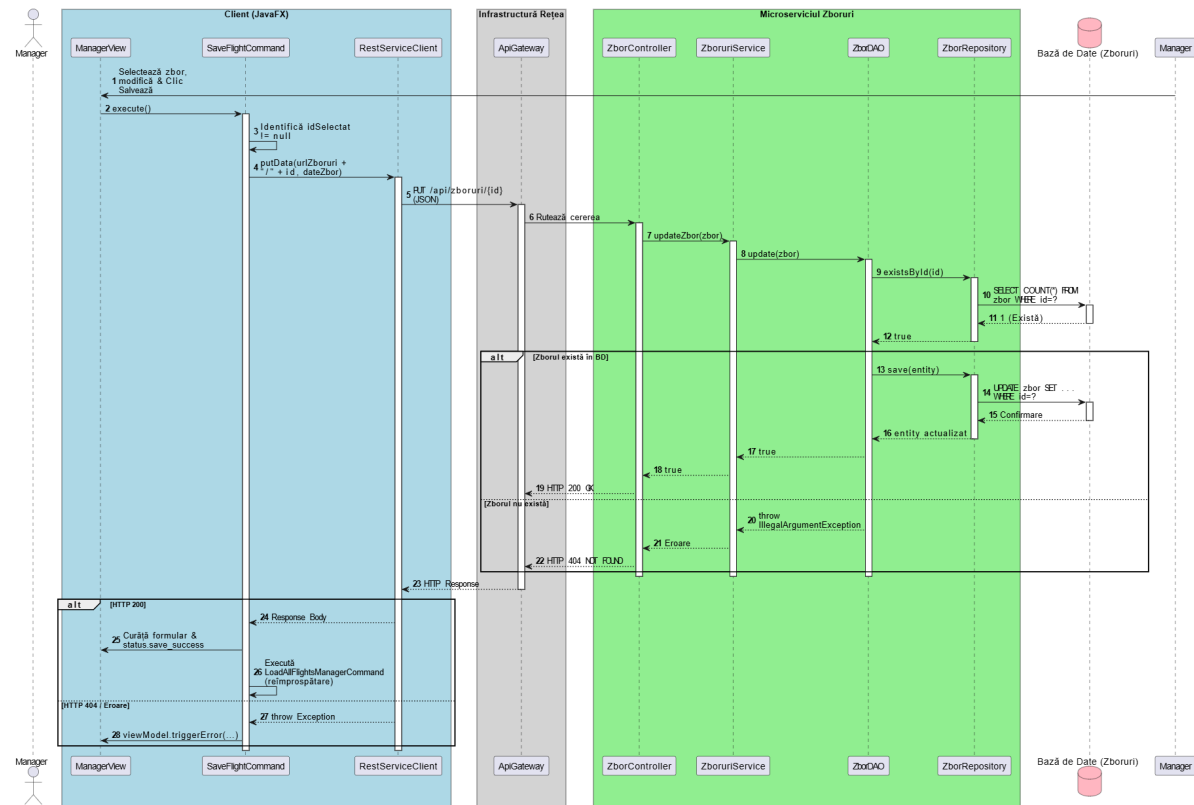


Figura 13: Diagrama de secvență: Actualizarea unui zbor existent

Descrierea fluxului: Procesul este similar cu adăugarea, însă este declanșat prin selectarea prealabilă a unui zbor din tabel. Comanda detectează prezența ID-ului și trimite un apel PUT la ruta `/api/zboruri/{id}`. Microserviciul verifică existența înregistrării și execută un UPDATE în baza de date. La primirea statusului 200 OK, aplicația client își resetează starea vizuală și reîncarcă datele actualizate din backend.

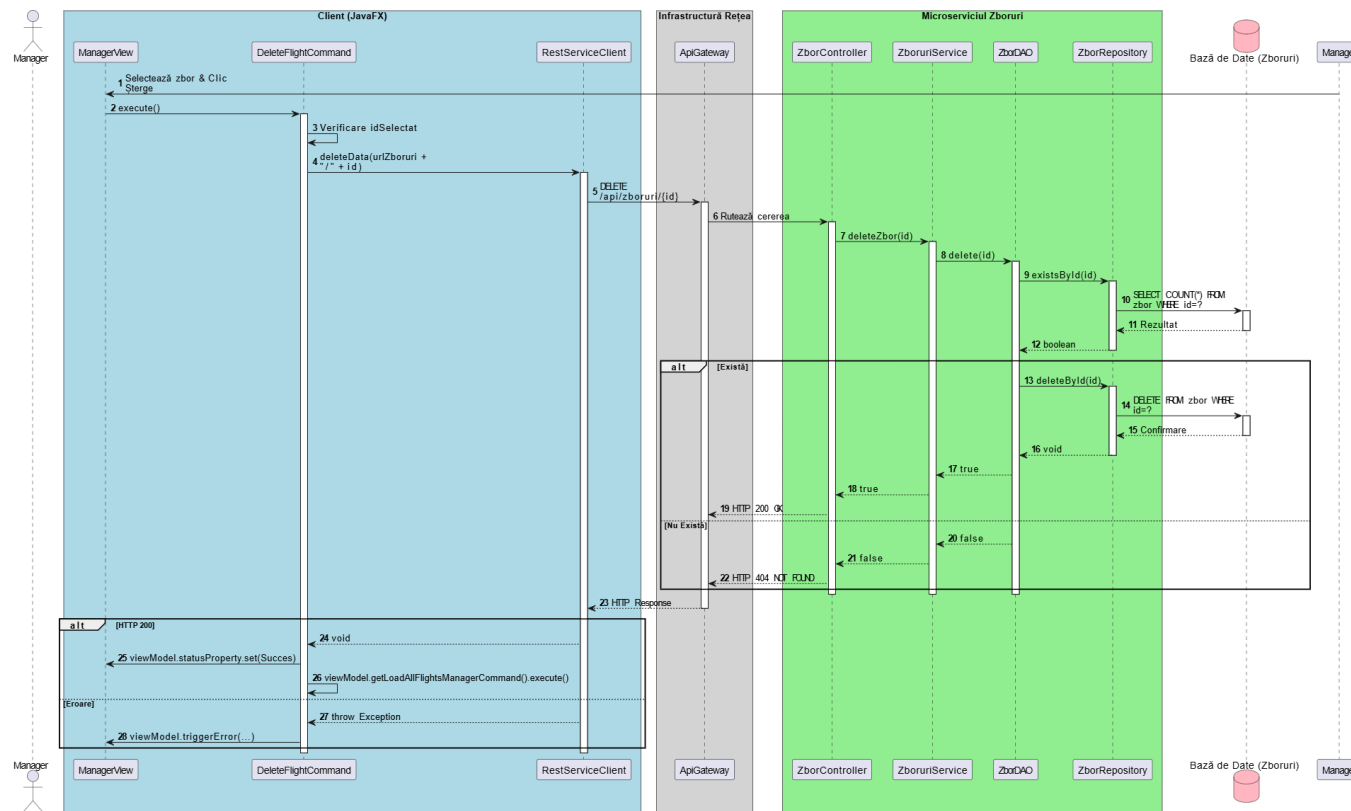


Figura 14: Diagrama de secvență: Ștergerea unui zbor

Descrierea fluxului: Managerul selectează un zbor și inițiază ștergerea. `DeleteFlightCommand` trimite o cerere `DELETE` ce conține ID-ul resursei. API Gateway rutează cererea la microserviciul corespunzător, care, după ce confirmă existența zborului, execută comanda de ștergere fizică sau logică în baza de date. Clientul JavaFX notifică utilizatorul printr-un mesaj de succes și forțează reîmprospătarea listei.

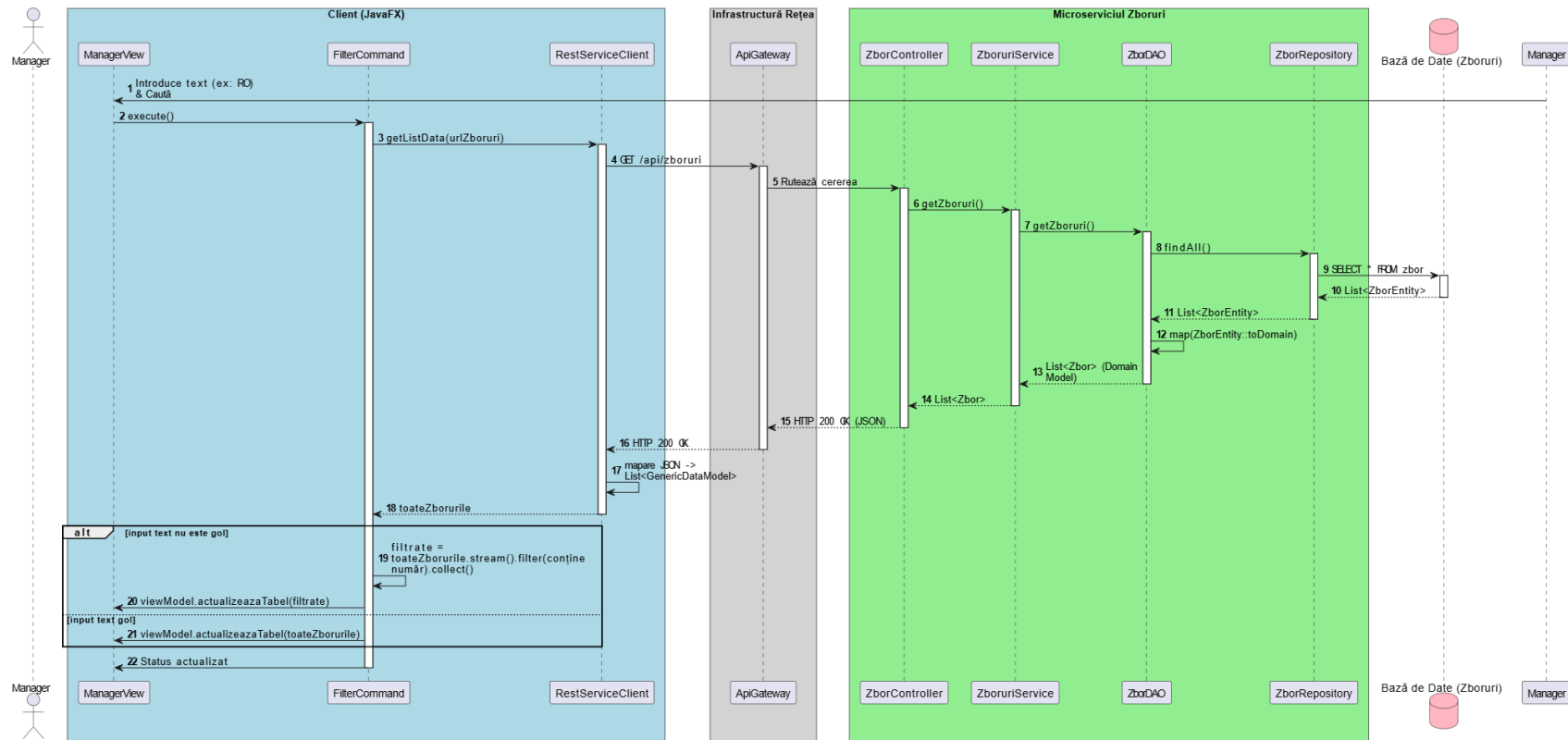


Figura 15: Diagrama de secvență: Căutarea și filtrarea zborurilor

Descrierea fluxului: Pentru a oferi o interacțiune fluidă (fără latență de rețea la fiecare tastare), **FilterCommand** preia inițial lista completă a zborurilor de la server printr-un apel **GET**. Când managerul introduce un text în câmpul de căutare, filtrarea se realizează strict la nivel local (în memoria clientului), utilizând Java Streams. Tabelul este actualizat instantaneu afișând doar înregistrările care conțin șirul de caractere introdus.

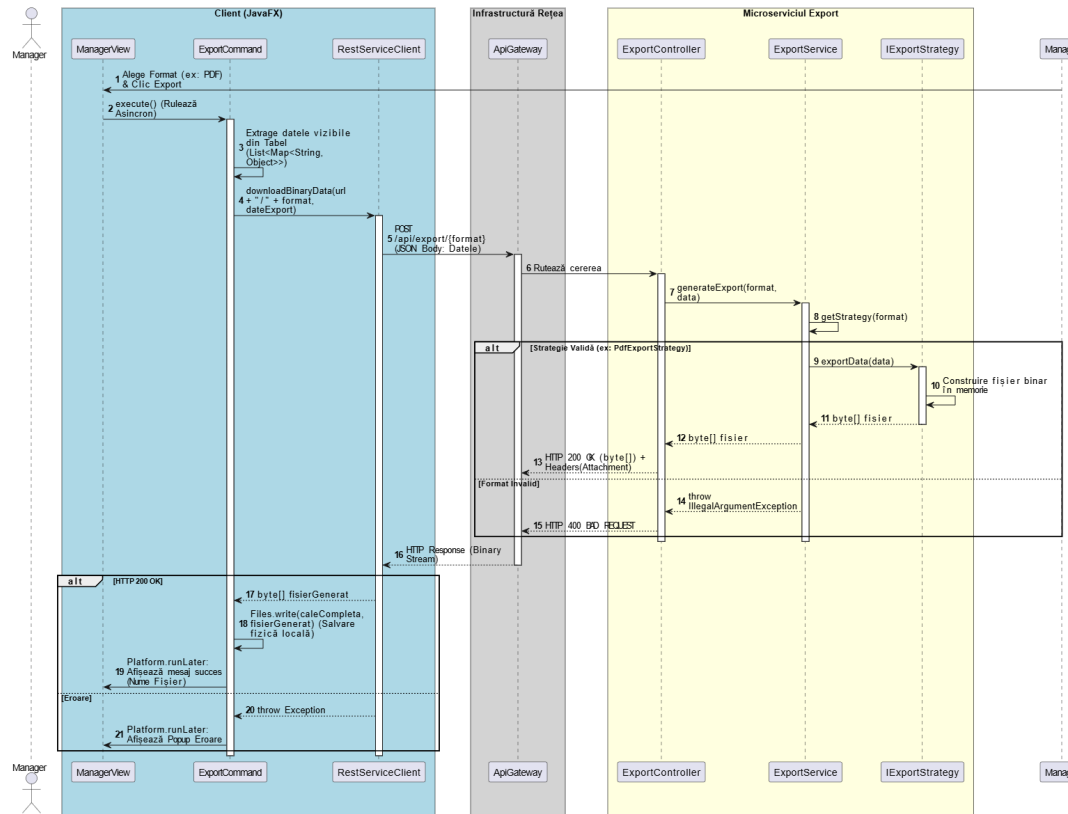


Figura 16: Diagrama de secvență: Exportul listei de zboruri

Descrierea fluxului: Funcționalitatea de export rulează asincron pentru a nu bloca interfața grafică. Clientul extrage datele vizibile curent în tabel și le trimite (ca array JSON) printr-un request POST către Microserviciul Export, specificând formatul dorit în URL (ex. PDF, CSV). Serviciul de export aplică design pattern-ul *Strategy* pentru a delega generarea corectă a documentului. Rezultatul este returnat ca un flux de octeți (`byte[]`), pe care aplicația îl scrie fizic pe disc.

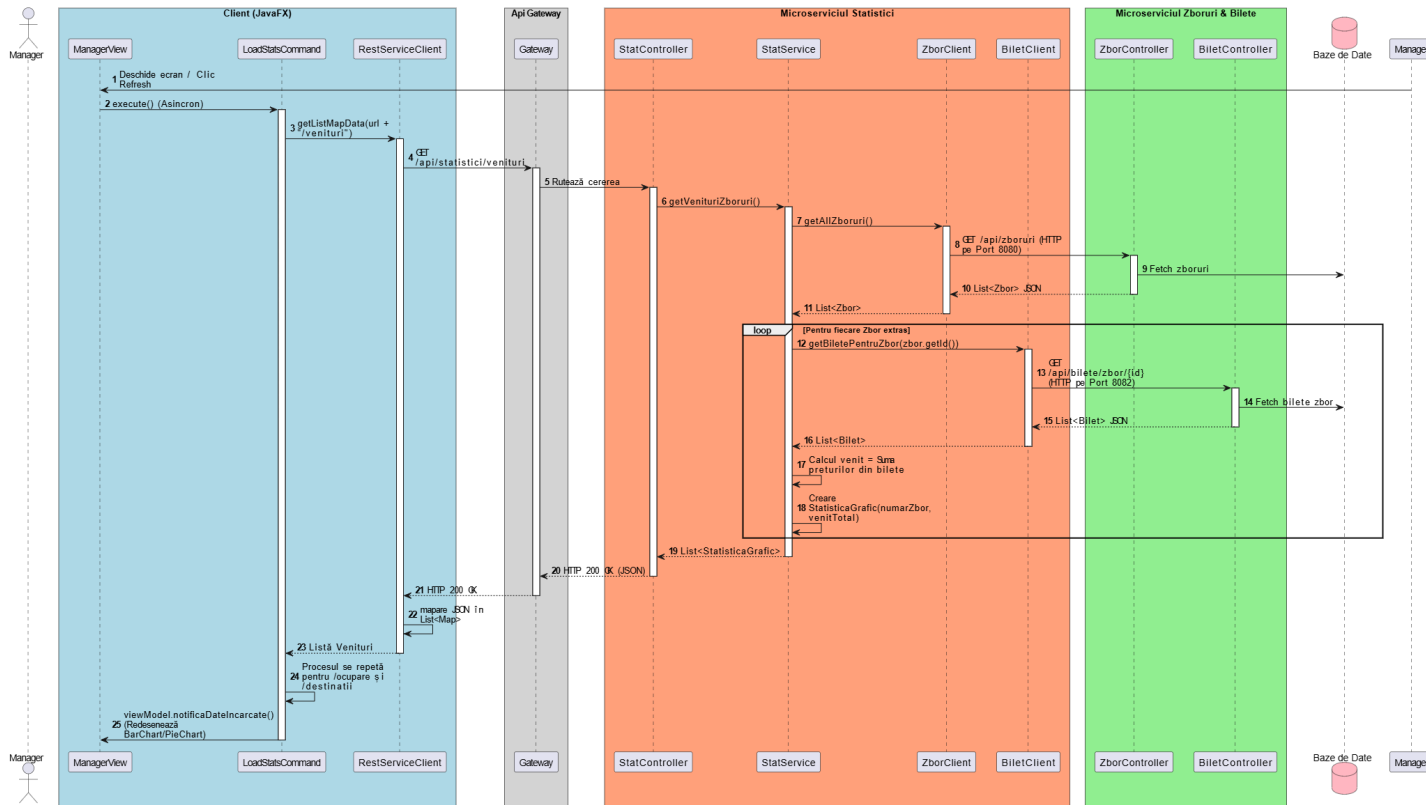


Figura 17: Diagrama de secvență: Vizualizarea statisticilor și agregarea datelor

Descrierea fluxului: La deschiderea ecranului de statistici, clientul face o cerere către **Microserviciul Statistici**. Acest serviciu are rol de orchestrator: nu posedă o bază de date proprie, ci comunică intern, via HTTP, cu **Microserviciul Zboruri** (pentru a obține lista rutelor) și cu **Microserviciul Bilete** (pentru a obține detaliile vânzărilor fiecărui zbor). Serviciul agreghează aceste date (ex. calculează suma veniturilor) și returnează un JSON consolidat clientului, care redesenează dinamic graficele de tip BarChart și PieChart.

1.5 Utilizator tip Administrator

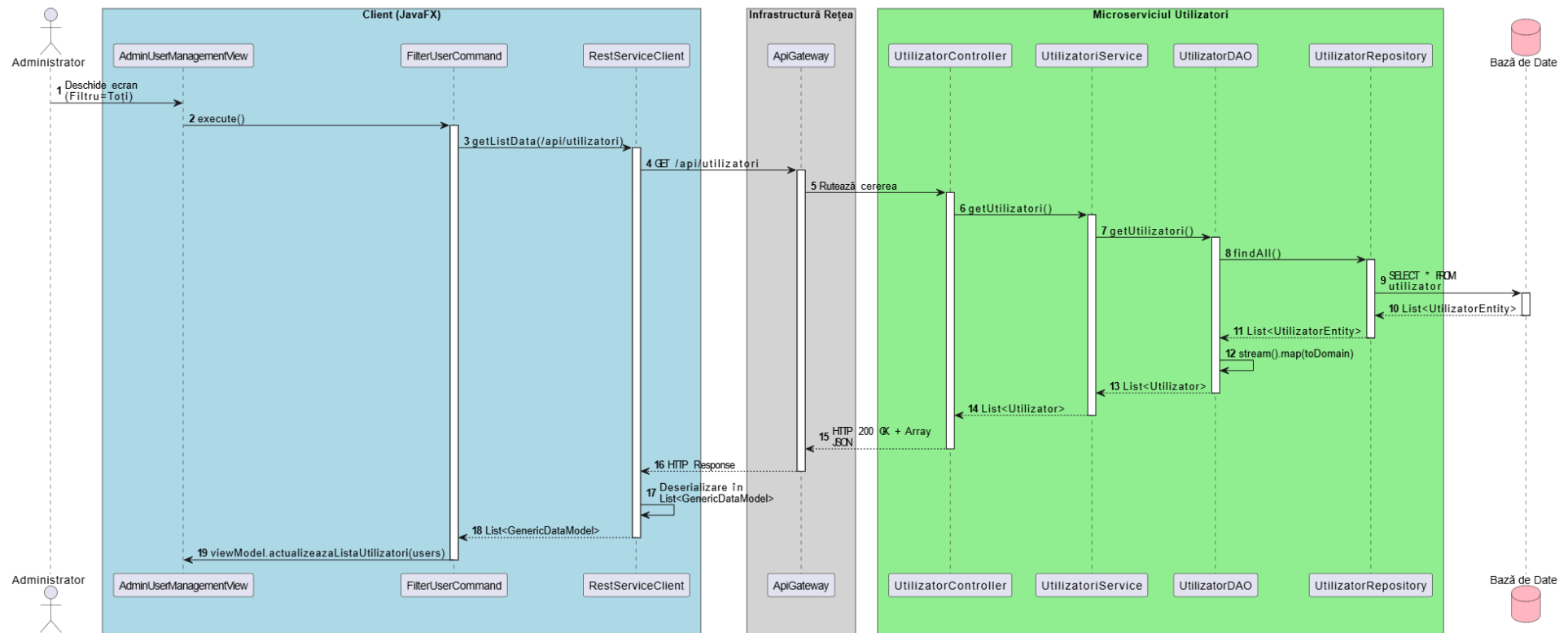


Figura 18: Diagrama de secvență: Vizualizarea listei de utilizatori

Descrierea fluxului: La deschiderea panoului de administrare, comanda trimite un apel GET către ruta `/api/utilizatori`. API Gateway rutează cererea la Microserviciul Utilizatori, care interoghează baza de date pentru a obține toți utilizatorii activi. Entitățile sunt mapate la modele de domeniu, serializate în JSON și returnate aplicației client, unde populează tabelul central.

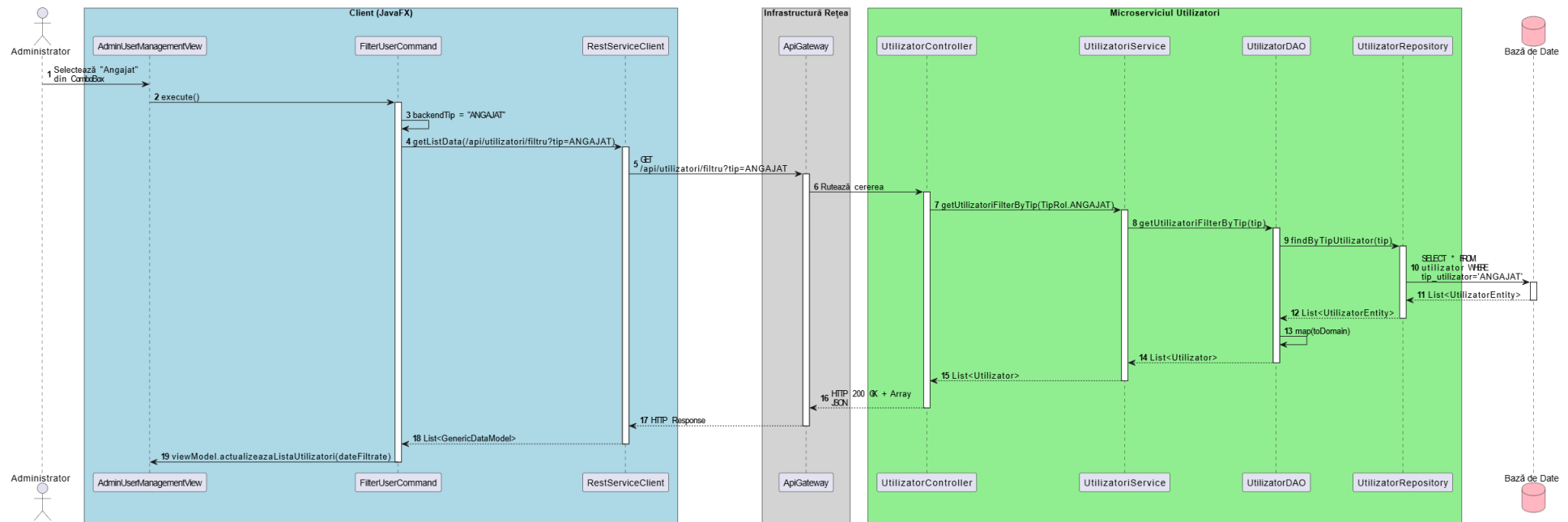


Figura 19: Diagrama de secvență: Filtrarea utilizatorilor după rol

Descrierea fluxului: Atunci când administratorul selectează un rol specific din meniul drop-down (ex. „Angajat”), se lansează un apel `GET` cu parametrul de query corespunzător (`?tip=ANGAJAT`). Backend-ul aplică filtrul direct la nivelul bazei de date (printr-o interogare JPA customizată) și returnează doar lista filtrată, optimizând astfel transferul de date din rețea.

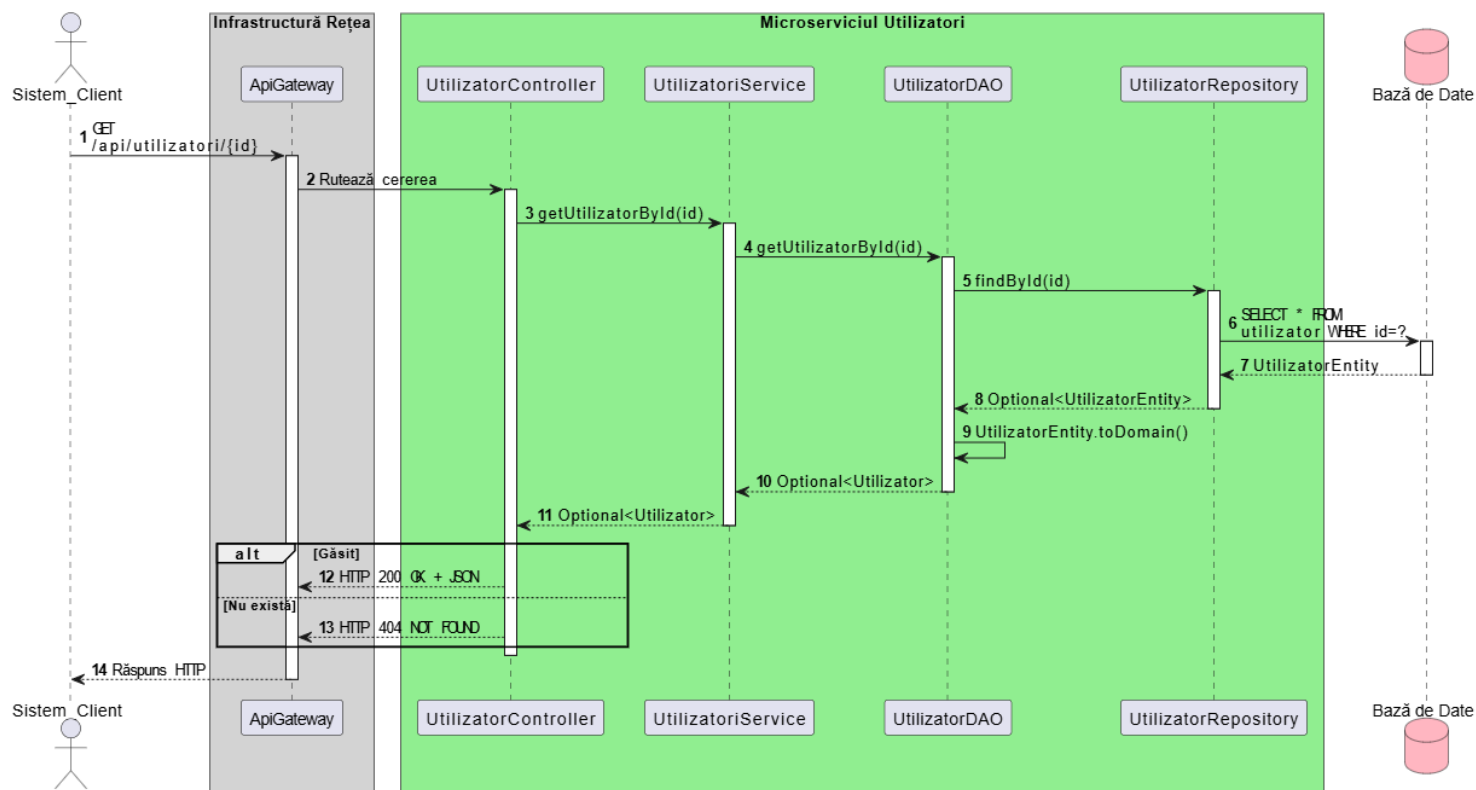


Figura 20: Diagrama de secvență: Căutarea unui utilizator specific (după ID)

Descrierea fluxului: Sistemul poate extrage detaliile unui singur utilizator printr-un apel `GET` pe ruta `/api/utilizatori/{id}`. Microserviciul caută înregistrarea utilizând comanda `findById`. Dacă entitatea este găsită, returnează `HTTP 200 OK` împreună cu obiectul JSON mapat; în caz contrar, sistemul răspunde cu `HTTP 404 NOT FOUND`.

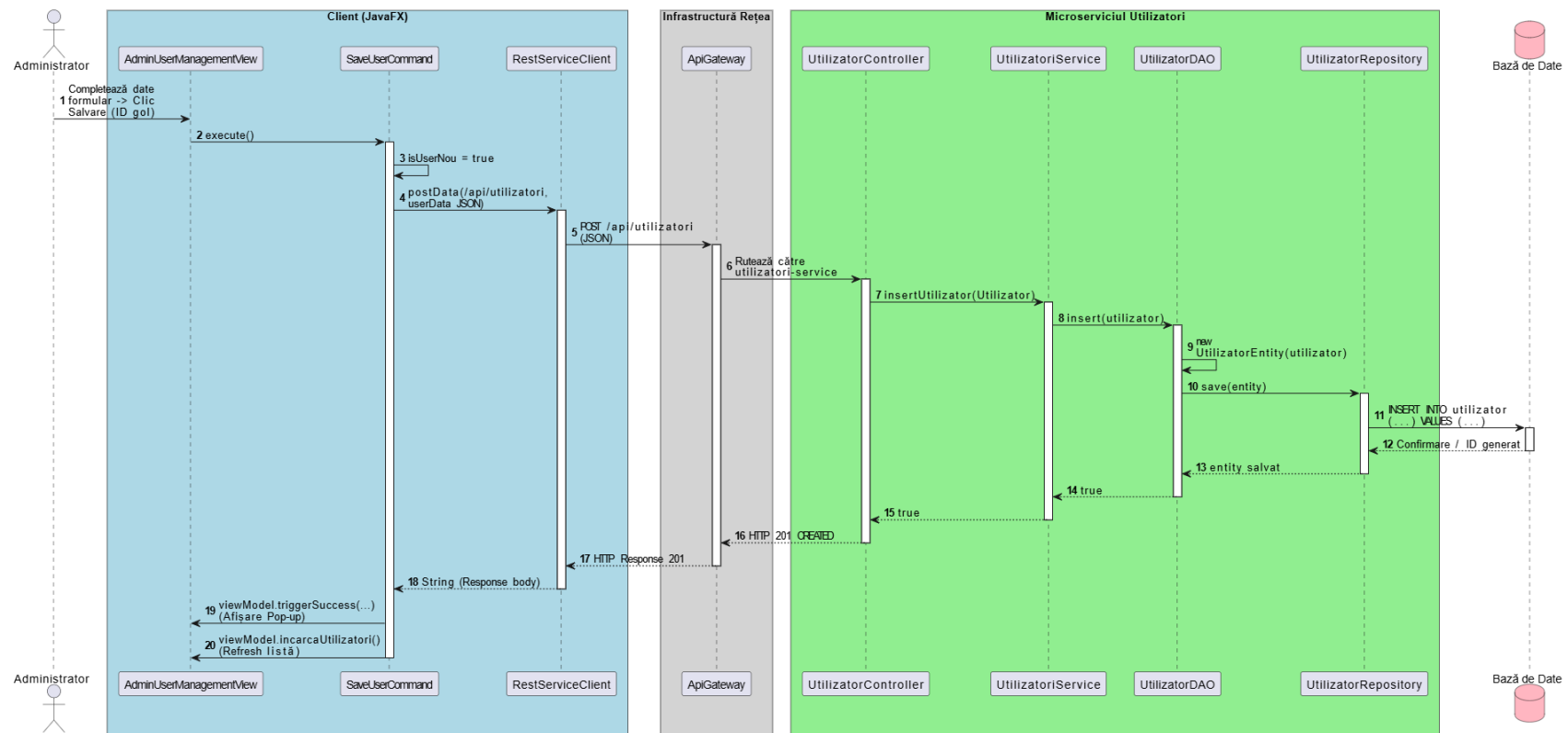


Figura 21: Diagrama de secvență: Adăugarea unui nou cont de utilizator

Descrierea fluxului: Pentru crearea unui cont nou, administratorul completează datele și apasă „Salvare”. Comanda detectează că ID-ul este gol (`isUserNou = true`) și inițiază un apel POST. După inserarea reușită în baza de date (`INSERT INTO utilizator`), clientul afișează un pop-up de succes și declanșează reîncărcarea listei din fundal pentru a afișa noul angajat.

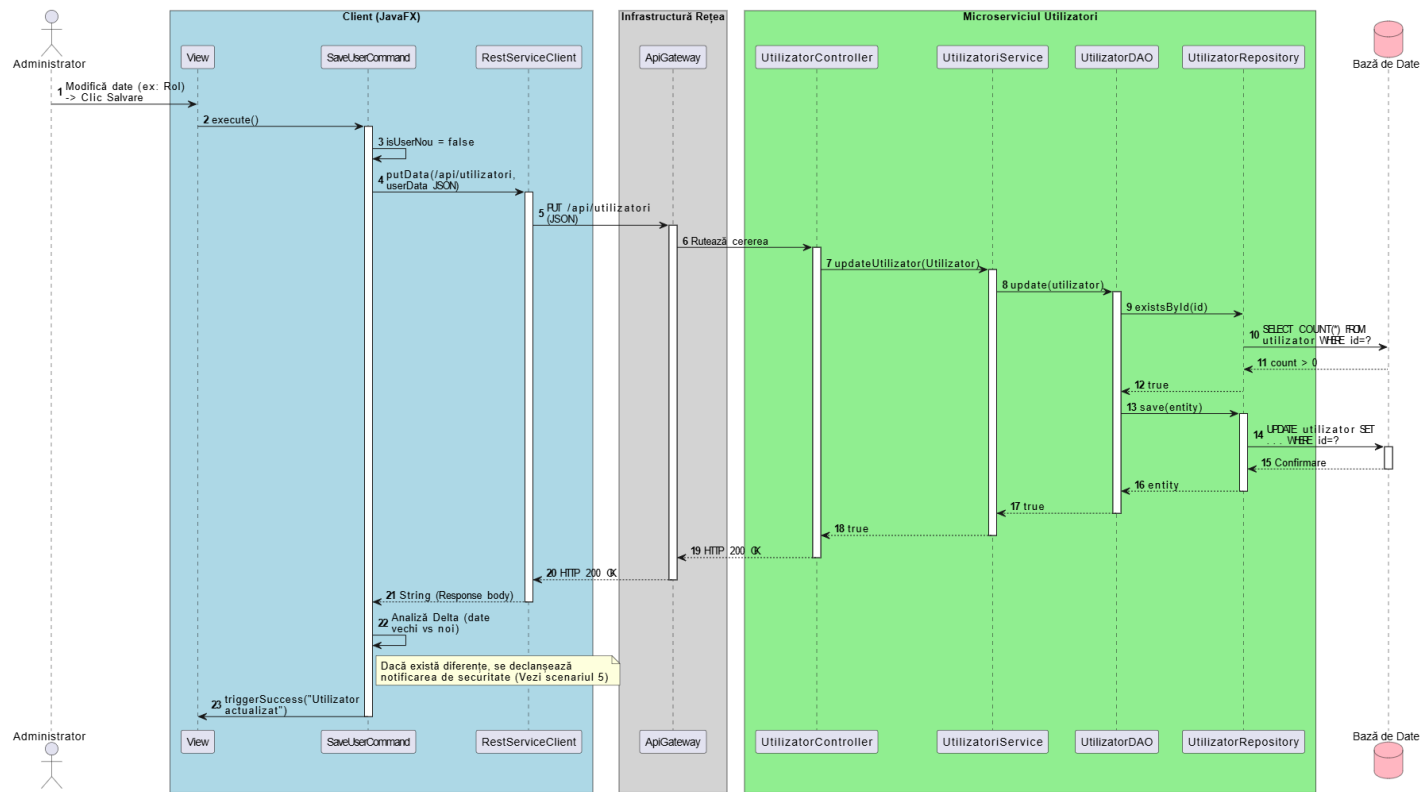


Figura 22: Diagrama de secvență: Actualizarea datelor unui cont existent

Descrierea fluxului: Când se editează un cont (`isUserNou = false`), se folosește metoda PUT. Înregistrarea este actualizată pe server, iar clientul primește răspunsul de confirmare. Suplimentar, clientul face o analiză de tip *Delta* (compară datele vechi cu cele noi). Dacă observă că s-au schimbat informații de securitate (ex. modificarea rolului în sistem), declanșează automat fluxul de notificare.

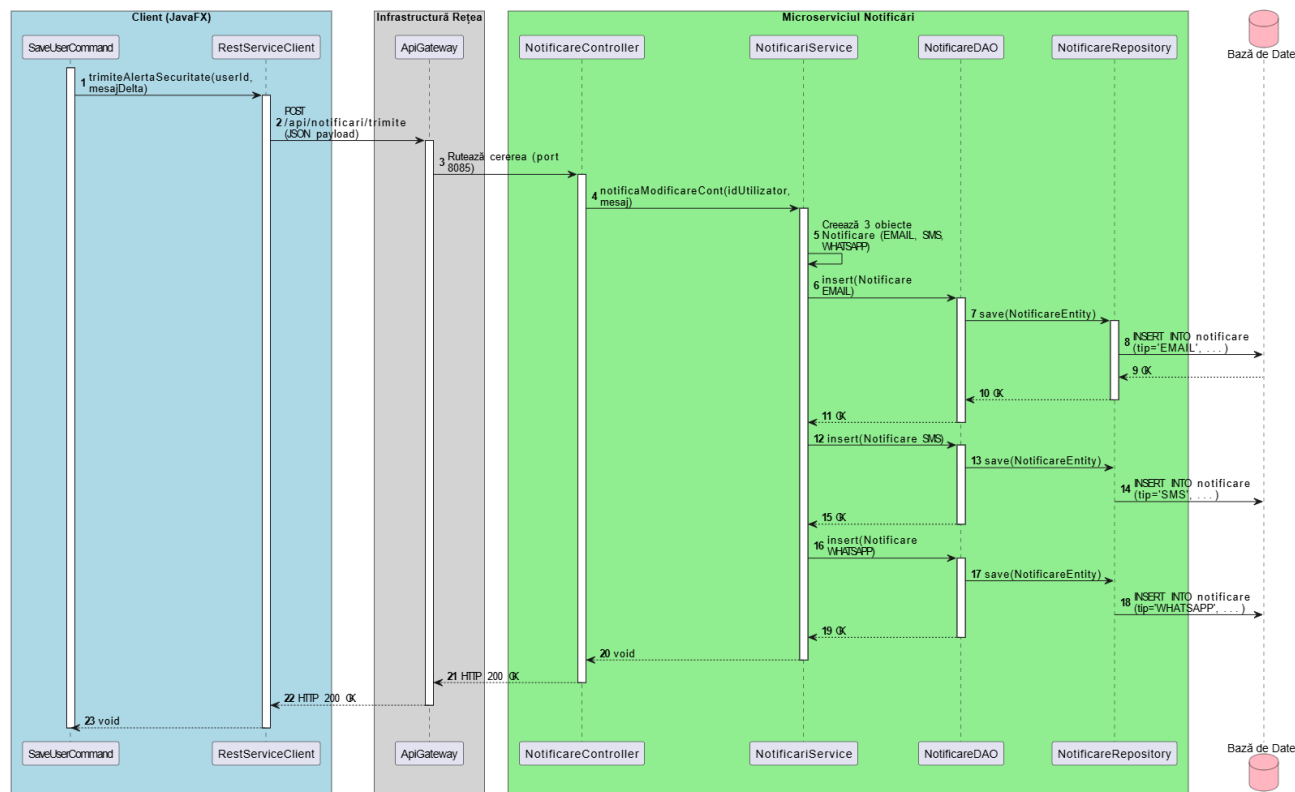


Figura 23: Diagrama de secvență: Trimiterea notificărilor de securitate pe mai multe canale

Descrierea fluxului: Apelul POST către Microserviciul Notificări este declanșat pentru a trimite alerte. Serviciul primește mesajul și creează asincron trei entități distincte de notificare în baza de date, corespunzătoare canalelor de comunicare suportate: EMAIL, SMS și WHATSAPP. Acestea pot fi ulterior procesate și expediate fizic de un job (cron) intern.

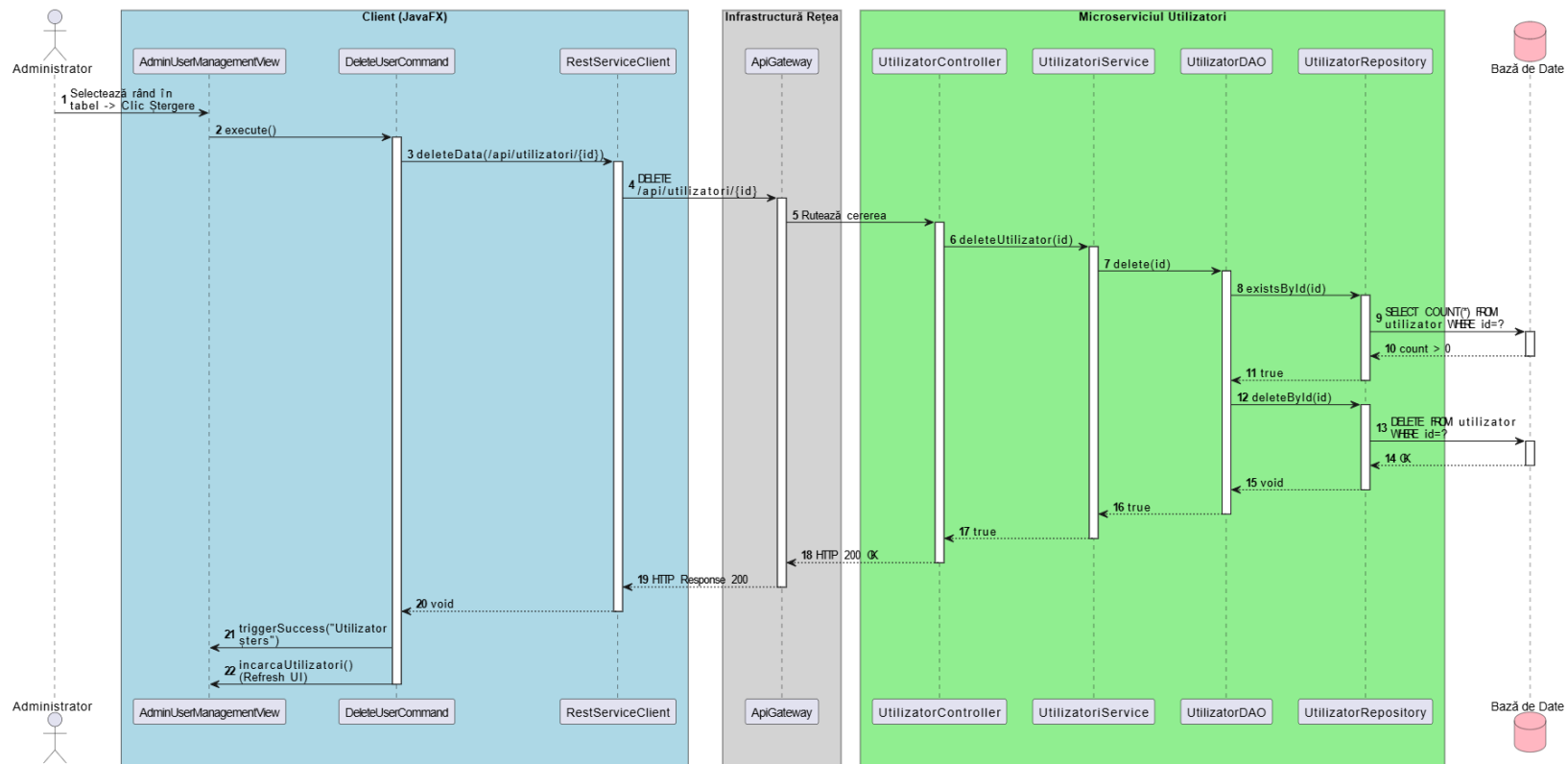


Figura 24: Diagrama de secvență: Ștergerea unui cont de utilizator

Descrierea fluxului: Administratorul selectează un rând din tabel și apasă butonul de ștergere. Comanda trimite un apel DELETE incluzând ID-ul utilizatorului în calea URL-ului. După ce repository-ul JPA execută instrucțiunea de DELETE pe server, răspunsul de HTTP 200 OK semnalează aplicației JavaFX să șteargă utilizatorul din interfața locală și să o reîmprospăteze.

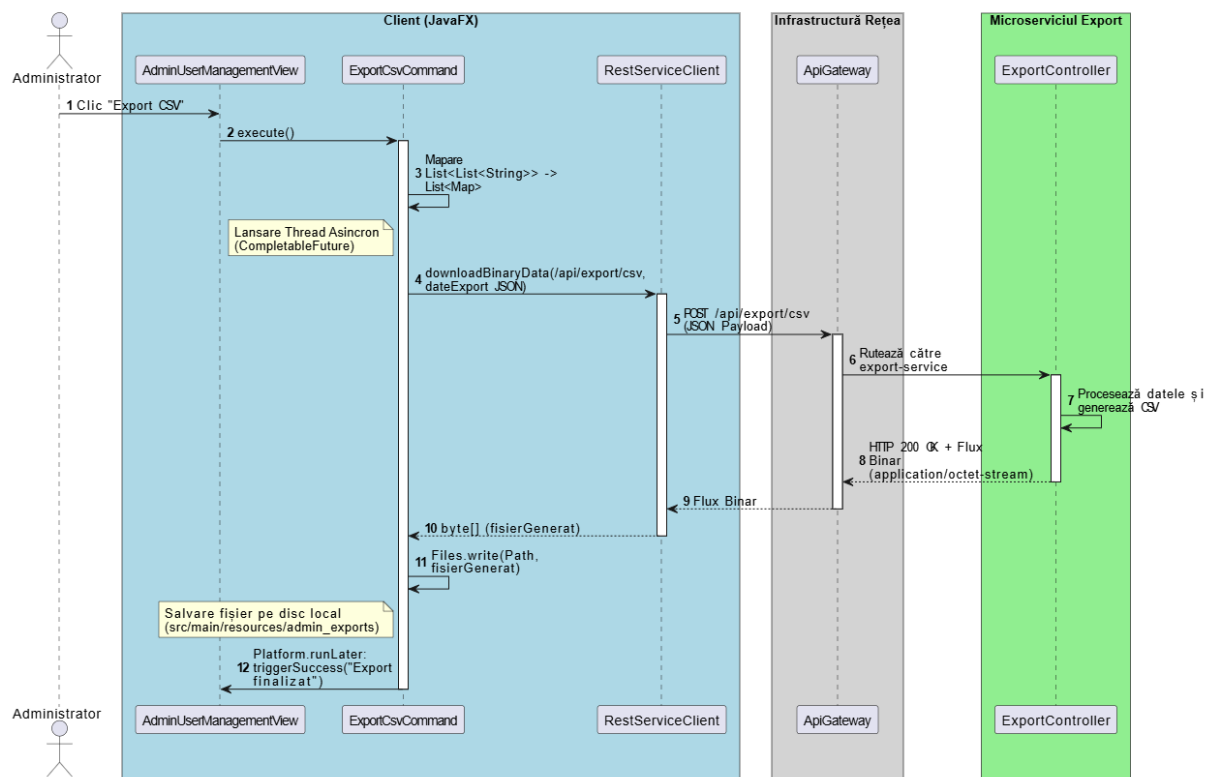


Figura 25: Diagrama de secvență: Exportul listei de utilizatori în format CSV

Descrierea fluxului: Lansat pe un fir de execuție separat (utilizând `CompletableFuture`) pentru a nu bloca interfața grafică, procesul preia datele utilizatorilor vizibili în tabel și le trimite printr-un `POST` către **Microserviciul Export**. Acesta din urmă procesează datele brute, generează un document CSV și îl returnează sub formă de flux binar (`application/octet-stream`), pe care aplicația îl descarcă pe mașina locală.